

Báo cáo bài tập thực hành tuần 7

Họ và tên: Nguyễn Văn Phúc Huy
MSSV: 23110163
Lớp: 23TTH (Chiều thứ 6, ca 1)

Viết chương trình để thực thi thuật toán hierarchical sử dụng phương pháp hợp nhất Bottom-up với single-linkage, complete-linkage và average-linkage.

```
class HierarchicalClustering:
    def __init__(self, data, K, linkage='single'):
        self.data = data.values.tolist()
        self.col = data.columns
        self.N = len(data)
        self.K = K
        self.linkage = linkage
        self.clusters = self.init_clusters()

    def init_clusters(self):
        return {data_id: [data_point] for data_id, data_point in enumerate(self.data)}

    # 1. Single-linkage
    def single_link(self, ci, cj):
        return np.min([np.linalg.norm(np.array(vi)-np.array(vj)) for vi in ci for vj in cj])

    # 2. Complete-linkage
    def complete_link(self, ci, cj):
        return np.max([np.linalg.norm(np.array(vi)-np.array(vj)) for vi in ci for vj in cj])

    # 3. Average-linkage
    def average_link(self, ci, cj):
        return np.mean([np.linalg.norm(np.array(vi)-np.array(vj)) for vi in ci for vj in cj])

    def calculate_distance(self, ci, cj):
        if self.linkage == 'single':
            return self.single_link(ci, cj)
        elif self.linkage == 'complete':
            return self.complete_link(ci, cj)
        elif self.linkage == 'average':
            return self.average_link(ci, cj)
        else:
            raise ValueError("Tham số linkage phải là 'single', 'complete' hoặc 'average'")

    def find_closest_clusters(self):
        clusters_ids = list(self.clusters.keys())

        cluster_combinations = [(clusters_ids[i], clusters_ids[j])
                                for i in range(len(clusters_ids)-1)
                                for j in range(i+1, len(clusters_ids))]

        distances = [self.calculate_distance(self.clusters[ci], self.clusters[cj])
                     for ci, cj in cluster_combinations]

        closest_index = np.argmin(distances)
        closest_clusters = cluster_combinations[closest_index]

        return closest_clusters

    def merge_clusters(self, ci_id, cj_id):
        new_clusters = {0: self.clusters[ci_id] + self.clusters[cj_id]}
        new_cluster_id = 1

        for cluster_id, points in self.clusters.items():
            if cluster_id != ci_id and cluster_id != cj_id:
                new_clusters[new_cluster_id] = points
                new_cluster_id += 1
```

```
return new_clusters
```

```
def run(self):
    while len(self.clusters.keys()) > self.K:
        closest_clusters = self.find_closest_clusters()
        self.clusters = self.merge_clusters(*closest_clusters)

def plot_clusters(self):
    cluster_ids = list(self.clusters.keys())
    num_clusters = len(cluster_ids)
    colors = sns.color_palette('rainbow', n_colors=num_clusters)
    fig, ax = plt.subplots()

    for i, cluster_id in enumerate(cluster_ids):
        points = self.clusters[cluster_id]
        x = [point[0] for point in points]
        y = [point[1] for point in points]
        ax.scatter(x, y, color=colors[i], label=f'Cluster {cluster_id}')

    ax.set_xlabel(self.col[0])
    ax.set_ylabel(self.col[1])
    ax.set_title(f'Hierarchical Clustering ({self.linkage}-linkage)')

    ax.legend()
    plt.show()
```

Class `HierarchicalClustering` đã được định nghĩa để thực thi thuật toán hierarchical clustering với ba phương pháp hợp nhất: single-linkage, complete-linkage và average-linkage. Hoạt động như sau:

- Hàm `__init__` khởi tạo đối tượng với dữ liệu, số cột, n (số lượng mẫu), số lượng cụm K và phương pháp hợp nhất.

```
def __init__(self, data, K, linkage='single'):
    self.data = data.values.tolist()
    self.col = data.columns
    self.N = len(data)
    self.K = K
    self.linkage = linkage
    self.clusters = self.init_clusters()
```

- Hàm `init_clusters` khởi tạo các cụm ban đầu, mỗi mẫu là một cụm riêng biệt.

```
def init_clusters(self):
    return {data_id: [data_point] for data_id, data_point in enumerate(self.data)}
```

- Các hàm `single_link`, `complete_link` và `average_link` tính toán khoảng cách giữa hai cụm theo phương pháp tương ứng.

- `single_link` tính khoảng cách nhỏ nhất giữa các điểm của hai cụm.

```
def single_link(self, ci, cj):
    return np.min([np.linalg.norm(np.array(vi)-np.array(vj)) for vi in ci for vj in cj])
```

- `complete_link` tính khoảng cách lớn nhất giữa các điểm của hai cụm.

```
def complete_link(self, ci, cj):
    return np.max([np.linalg.norm(np.array(vi)-np.array(vj)) for vi in ci for vj in cj])
```

- `average_link` tính khoảng cách trung bình giữa các điểm của hai cụm.

```
def average_link(self, ci, cj):
    return np.mean([np.linalg.norm(np.array(vi)-np.array(vj)) for vi in ci for vj in cj])
```

- Hàm `calculate_distance` chọn phương pháp hợp nhất dựa trên tham số `linkage` được cung cấp.

```
def calculate_distance(self, ci, cj):
    if self.linkage == 'single':
        return self.single_link(ci, cj)
    elif self.linkage == 'complete':
        return self.complete_link(ci, cj)
    elif self.linkage == 'average':
        return self.average_link(ci, cj)
    else:
        raise ValueError("Tham số linkage phải là 'single', 'complete' hoặc 'average'")
```

- Hàm `find_closest_clusters` tìm cặp cụm gần nhất dựa trên khoảng cách được tính toán. Hoạt động bằng cách:
 - Tạo danh sách các cặp cụm có thể.
 - Tính khoảng cách cho mỗi cặp cụm.
 - Xác định cặp cụm có khoảng cách nhỏ nhất và trả về cặp đó.

```

def find_closest_clusters(self):
    clusters_ids = list(self.clusters.keys())

    cluster_combinations = [(clusters_ids[i], clusters_ids[j])
                             for i in range(len(clusters_ids)-1)
                             for j in range(i+1, len(clusters_ids))]

    distances = [self.calculate_distance(self.clusters[ci], self.clusters[cj])
                  for ci, cj in cluster_combinations]

    closest_index = np.argmin(distances)
    closest_clusters = cluster_combinations[closest_index]

    return closest_clusters

```

- Hàm `merge_clusters` hợp nhất hai cụm được chỉ định thành một cụm mới và cập nhật danh sách các cụm. Hoạt động bằng cách:
 - Tạo một từ điển mới để lưu trữ các cụm sau khi hợp nhất.
 - Hợp nhất hai cụm được chỉ định thành một cụm mới với ID 0.
 - Thêm các cụm còn lại vào từ điển mới với ID mới.

```

def merge_clusters(self, ci_id, cj_id):
    new_clusters = {0: self.clusters[ci_id] + self.clusters[cj_id]}
    new_cluster_id = 1
    for cluster_id, points in self.clusters.items():
        if cluster_id != ci_id and cluster_id != cj_id:
            new_clusters[new_cluster_id] = points
            new_cluster_id += 1
    return new_clusters

```

- Hàm `run` thực thi thuật toán hierarchical clustering cho đến khi số lượng cụm còn lại bằng K. Hoạt động bằng cách:
 - Trong khi số lượng cụm lớn hơn K, tìm cặp cụm gần nhất và hợp nhất chúng.

```

def run(self):
    while len(self.clusters.keys()) > self.K:
        closest_clusters = self.find_closest_clusters()
        self.clusters = self.merge_clusters(*closest_clusters)

```

- Hàm `plot_clusters` vẽ biểu đồ phân cụm sau khi thuật toán đã hoàn thành. Hoạt động bằng cách:
 - Lấy danh sách ID của các cụm và số lượng cụm.
 - Tạo một biểu đồ phân tán với màu sắc khác nhau cho mỗi cụm.
 - Đặt nhãn cho trục x, trục y và tiêu đề biểu đồ.
 - Hiển thị biểu đồ.

```

def plot_clusters(self):
    cluster_ids = list(self.clusters.keys())
    num_clusters = len(cluster_ids)
    colors = sns.color_palette('rainbow', n_colors=num_clusters)
    fig, ax = plt.subplots()
    for i, cluster_id in enumerate(cluster_ids):
        points = self.clusters[cluster_id]
        x = [point[0] for point in points]
        y = [point[1] for point in points]
        ax.scatter(x, y, color=colors[i], label=f'Cluster {cluster_id}')
    ax.set_xlabel(self.col[0])
    ax.set_ylabel(self.col[1])
    ax.set_title(f'Hierarchical Clustering ({self.linkage}-linkage)')
    ax.legend()
    plt.show()

```

Single-linkage

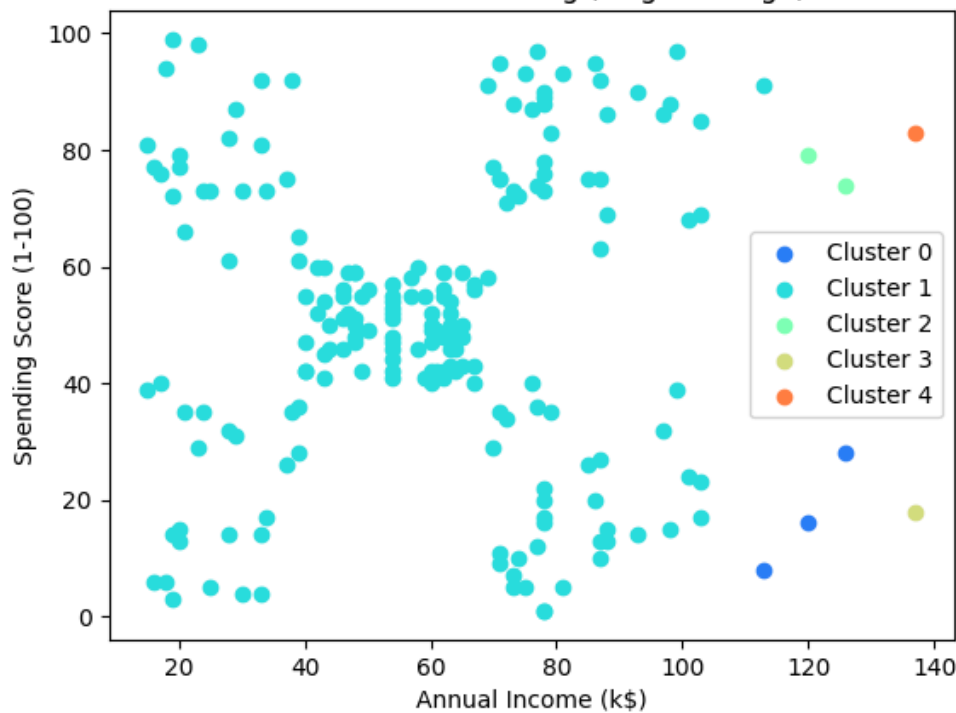
Code viết bằng côm

```

In [31]: hierarchical_clustering_single = HierarchicalClustering(selected_data, 5, linkage='single')
hierarchical_clustering_single.run()
hierarchical_clustering_single.plot_clusters()

```

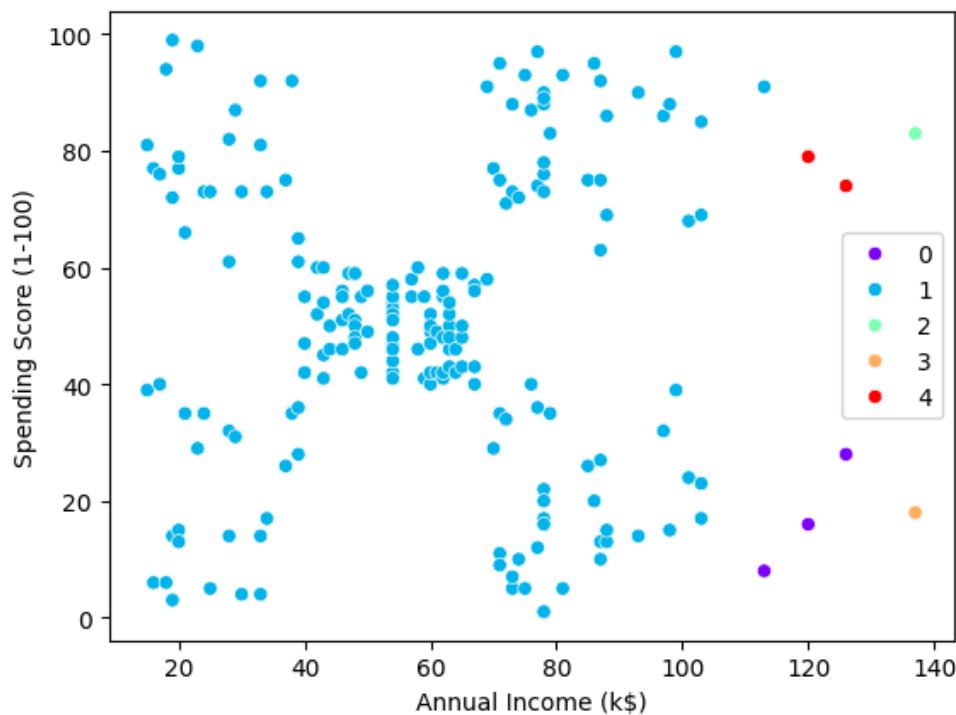
Hierarchical Clustering (single-linkage)



Code hàm có sẵn

[illegible]

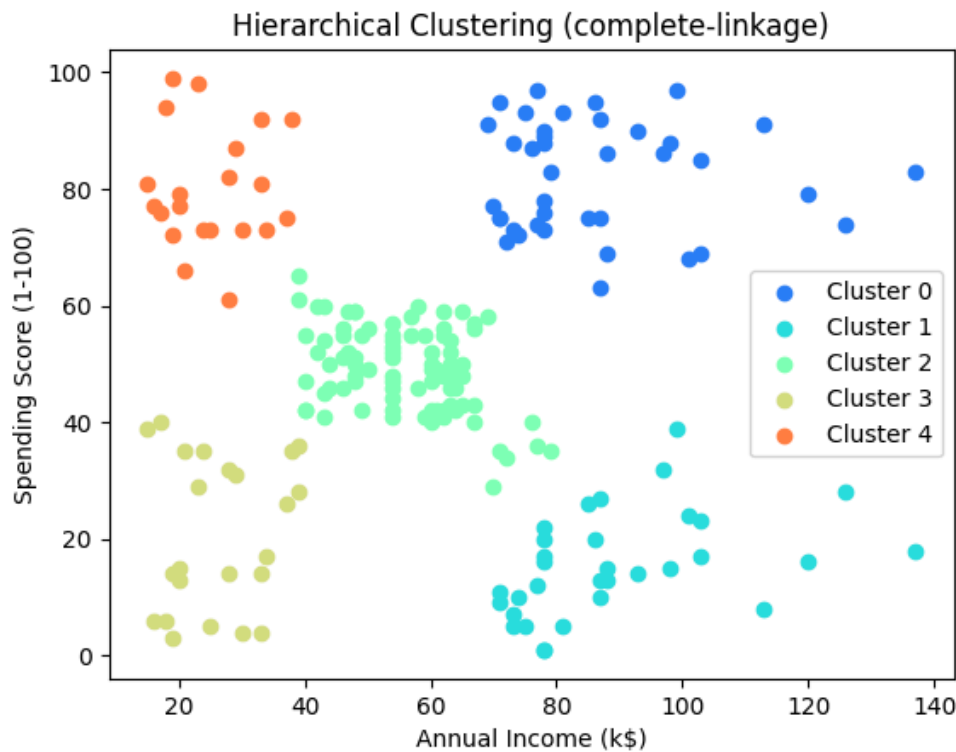
```
Out[32]: <Axes: xlabel='Annual Income (k$)', ylabel='Spending Score (1-100)'>
```



Complete-linkage

Code viết bằng cơm

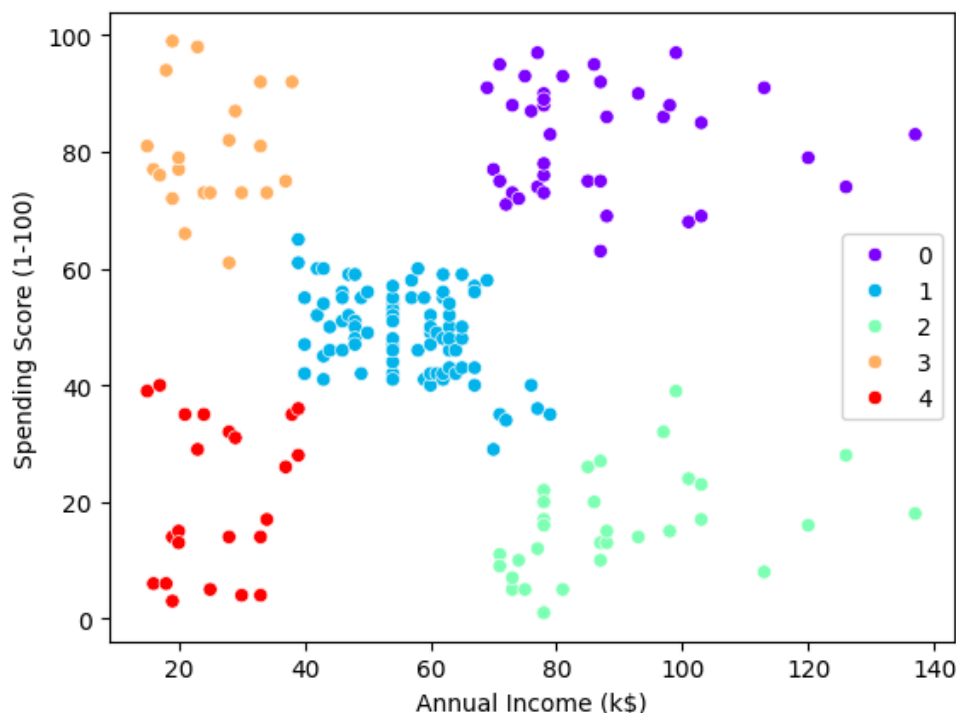
```
In [33]: hierarchical_clustering_complete = HierarchicalClustering(selected_data, 5, linkage='complete')
hierarchical_clustering_complete.run()
hierarchical_clustering_complete.plot_clusters()
```



Code hàm có sẵn

[illegible]

```
Out[34]: <Axes: xlabel='Annual Income (k$)', ylabel='Spending Score (1-100)'>
```



Code viết bằng cơm

Hierarchical Clustering (average-linkage)

This scatter plot displays the results of hierarchical clustering using average-linkage on the 'Spending Score (1-100)' (Y-axis) versus 'Annual Income (k\$)' (X-axis) data. The data points are grouped into five distinct clusters, each represented by a different color:

- Cluster 0 (Blue):** Located in the lower right quadrant, representing individuals with high annual income (approx. 70k to 140k) and low spending scores (approx. 0 to 40).
- Cluster 1 (Cyan):** Located in the center-left, representing individuals with moderate annual income (approx. 15k to 65k) and moderate spending scores (approx. 5 to 60).
- Cluster 2 (Light Green):** Located in the upper center, representing individuals with moderate annual income (approx. 70k to 110k) and high spending scores (approx. 60 to 95).
- Cluster 3 (Yellow-Green):** Located in the upper left, representing individuals with low annual income (approx. 15k to 40k) and high spending scores (approx. 60 to 100).
- Cluster 4 (Orange):** Located in the far right, representing individuals with very high annual income (approx. 120k to 140k) and high spending scores (approx. 70 to 85).

[illegible]

```
Out[35]: <Axes: xlabel='Annual Income (k$)', ylabel='Spending Score (1-100)'>
```

